

UD4.OPTIMIZACIÓN Y DOCUMENTACIÓN

UD4.3.CONTROL DE VERSIONES

INTRODUCCIÓN

El software está sometido a continuos cambios durante el desarrollo, pero también después de su entrega. Esto ha llevado a que se incluya la tarea de mantenimiento dentro de las fases de los ciclos de vida del desarrollo de una aplicación informática. Adicionalmente a esta tarea, surge otra labor de gestión igual de importante que se conoce como Gestión de la Configuración del Software (GCS) que tiene 4 objetivos fundamentales:

1. Identificar el cambio.
2. Controlar el cambio.
3. Garantizar que el cambio se implemente de manera adecuada.
4. Informar de los cambios aplicados a todos aquellos a los que pueda afectar.

INTRODUCCIÓN

La configuración del software está formada por los elementos de configuración (EC) que se han desarrollado y aprobado en algún momento del ciclo de vida del proyecto. Estos EC pueden ser de dos tipos:

- ▶ Básicos. (ej: código fuente de una clase)
- ▶ Agregados. Es un conjunto de EC básicos (ej: el código fuente de la aplicación completa).

Según avanza el proyecto, se crearán nuevas versiones de los EC. En términos informáticos y consultando la RAE, una versión es: «Cada una de las formas que adopta la relación de un suceso, el texto de una obra o la interpretación de un tema». Este concepto hace referencia a las diferentes formas que puede adoptar un EC a lo largo de su existencia.

Por tanto, una tarea correspondiente a la GCS es la identificar cada una de las versiones de cada EC.

INTRODUCCIÓN

La GCS es la encargada de gestionar los EC, identificándolos, controlándolos, informando sobre su estado y validando su contenido.

En la actualidad, la GCS se puede llevar a cabo mediante herramientas de control de versiones.

Toda herramienta de este tipo hace uso de un repositorio, que se puede definir como un almacén que contiene toda la información sobre un proyecto, lo que incluye, por tanto, todas las versiones de todos los EC que forman parte de la configuración del software.

La existencia de este repositorio posibilita el almacenamiento de las relaciones existentes entre los diferentes EC, lo cual es fundamental para conocer los impactos de los cambios realizados sobre el software.

Esto también permite conocer qué elementos del software se verán afectados , por ejemplo, por la realización de una modificación en una parte de un diagrama de clases.

Gestión de la Configuración del Software (GCS)

La GCS está regulada en el estándar IEEE 828-2005, consta de las siguientes actividades:

1. **Identificación de los EC:** se establece un método claro y consistente para identificar cada EC y cada una de sus versiones.
2. **Control de cambios:** el objetivo de esta actividad es asegurarse de que los cambios aceptados sobre un proyecto se incorporan a este sin causar problemas. Para ello, ante una petición de cambio, se analiza y evalúa su impacto, y se aprueba o no el cambio solicitado. Si se acepta el cambio, se genera una orden de cambio documentando el efecto del mismo y cómo se debe llevar a cabo. Se indican las restricciones que se deben respetar y los criterios para su revisión. Una vez implementado el cambio, se actualiza la versión de los EC afectados por el cambio.

Gestión de la Configuración del Software (GCS)

3. **Auditoría de configuración:** con la cual se comprueba si el cambio se implementó correctamente. Para esto, se dispone de dos medios:
 - ▶ revisiones técnicas formales: determinan si el o los EC cambiados son correctos desde el punto de vista técnico.
 - ▶ auditorías de configuración: controlan otros aspectos como si se realizó el cambio especificado en la orden de cambio, si se llevó a cabo la revisión técnica, si se siguieron los procedimientos de la GCS para reflejar el cambio, si los EC se actualizaron correctamente, si se informó del cambio convenientemente, etc...
4. **Generación de informes:** se debe documentar de manera adecuada el cambio que se ha realizado sobre la configuración del software para que el equipo de desarrollo sepa qué versión del EC deben utilizar, cuál está sujeto a peticiones de cambio y cuáles son los definitivos.

ENTORNOS DE UN SISTEMA

En el desarrollo de un sistema informático cabe diferenciar diferentes entornos:

- ▶ Entorno de desarrollo: Donde se lleva a cabo la programación
- ▶ Entorno de integración: Donde se lleva a cabo las pruebas de integración
- ▶ Entorno de producción: Donde se lleva a cabo la explotación del sistema informático

Un mismo desarrollo puede tener diferentes versiones según se esté en el entorno de desarrollo, de integración o de producción. Esta división de entornos puede requerir:

- ▶ Actualizar la versión del software: Consiste en trasladar la versión existente en el entorno de desarrollo hacia el entorno de producción, asegurando que los cambios que conlleva la nueva versión no afectan al funcionamiento de otros módulos o componentes del software ya desarrollados. De ahí que es necesario, dependiendo del alcance de los cambios, el realizar de nuevo pruebas de integración
- ▶ Volver a una versión anterior en caso de detectar fallos que implican cambios en el desarrollo y comportamientos anómalos del sistema.

CONTROL DE VERSIONES

Una **versión**, revisión o edición de un producto, es el estado en el que se encuentra dicho producto en un momento dado de su desarrollo o modificación.

Se llama **control de versiones** a la gestión de los diversos cambios que se realizan sobre los elementos de algún producto o una configuración del mismo. Los sistemas de control de versiones facilitan la administración de las distintas versiones de cada producto desarrollado, así como las posibles especializaciones realizadas (por ejemplo, para algún cliente en particular).

El control de versiones se realiza principalmente en la industria informática para controlar las distintas versiones del código fuente. Sin embargo, los mismos conceptos son aplicables a otros ámbitos como documentos, imágenes, sitios web, etcétera.

CONTROL DE VERSIONES

Aunque un sistema de control de versiones puede realizarse de forma manual, es muy aconsejable disponer de herramientas que faciliten esta gestión (CVS, Subversion, SourceSafe, ClearCase, Darcs, Bazaar, Plastic SCM, Git, Mercurial, Perforce...).

Un sistema de control de versiones debe proporcionar además un registro histórico de las acciones realizadas con cada elemento o conjunto de elementos (normalmente pudiendo volver o extraer un estado anterior del producto).

Procedimiento habitual:

1. El proyecto está disponible en un repositorio en un servidor.
2. Se realiza una copia del proyecto desde el repositorio al equipo local del Programador.
3. Se realizan los cambios necesarios sobre el código en local.
4. Se envían los cambios al servidor integrándolos de forma adecuada.

CONTROL DE VERSIONES - Conceptos

- **Repositorio.** Es el lugar designado para almacenar todos los datos y los cambios. Puede ser un sistema de archivos en un disco duro, un banco de datos, un servidor, etc. Puede ser:
 - local: Es un repositorio completo que tiene cada usuario en su máquina.
 - remoto: Es una referencia a cualquier otro repositorio que no sea el local.
- **Revisión o versión.** Una revisión es una versión concreta de los datos almacenados. Suelen ir numeradas. La última versión válida se denomina HEAD.
- **Etiquetar (tag).** Cuando se crea una versión concreta en un momento determinado del desarrollo de un proyecto se le pone una etiqueta, de forma que se pueda localizar y recuperar en cualquier momento.
- **Tronco (trunk).** Es la línea principal de desarrollo de un proyecto.
- **Rama (branch).** Las ramas son bifurcaciones en la línea de desarrollo de un proyecto. Se parte de una copia de archivos, carpetas o proyectos. A partir de ese momento, se trabaja en dos líneas de desarrollo. Suelen aparecer cuando hay que desarrollar alguna funcionalidad nueva o corregir algún error.

CONTROL DE VERSIONES - Conceptos

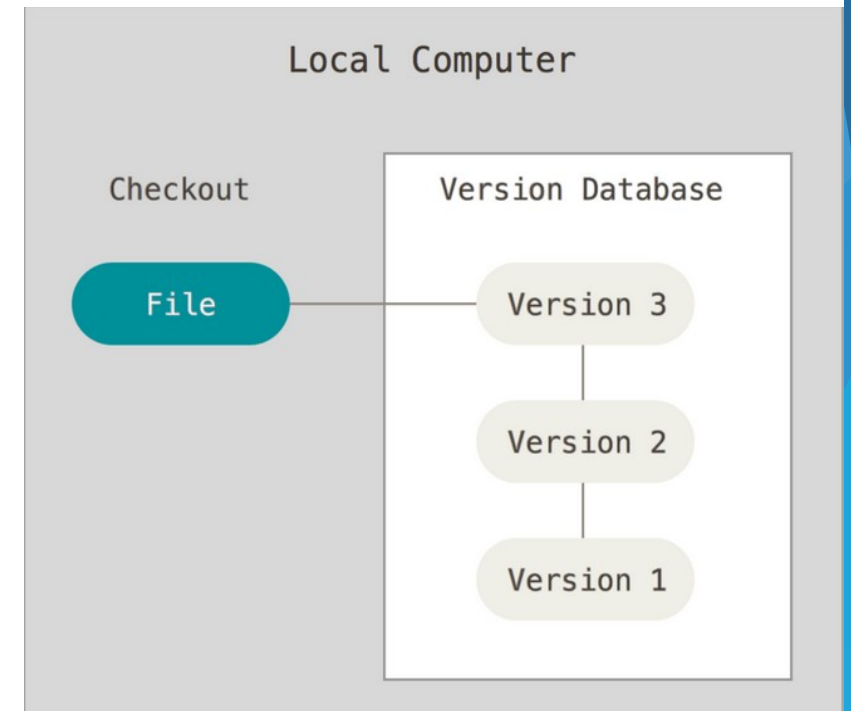
- **Desplegar (Checkout).** Crea una copia de trabajo del proyecto, o de archivos y carpetas del repositorio en el equipo local vinculada con el repositorio.
- **Confirmar (commit o check-in).** Se realiza commit cuando se confirman los cambios realizados en local para integrarlos al repositorio.
- **Exportación (export).** Similar a Checkout, pero en esta ocasión no vincula la copia con el repositorio. Es una copia limpia sin los metadatos de control de versiones.
- **Importación (import).** Es la subida de carpetas y archivos del equipo local al repositorio.
- **Actualizar (update).** Se realiza una actualización cuando se desea integrar los cambios realizados en el repositorio en la copia de trabajo local.

CONTROL DE VERSIONES - Conceptos

- **Fusión (merge).** Consiste en unir los cambios realizados sobre uno o varios archivos en una única revisión. Se produce cuando hay varias líneas de desarrollo separadas en ramas y en alguna etapa se necesitan fusionar los cambios hechos entre ramas o en una rama con la línea principal, o viceversa.
- **Conflicto.** Ocurre cuando dos usuarios crean una copia local (Checkout) de la misma versión de un archivo, uno de ellos realiza cambios y envía los cambios (commit) al repositorio, y el otro no actualiza (update) esos cambios y realiza cambios sobre el archivo e intenta enviar luego sus cambios al repositorio. En estos casos el usuario debe revisar lo ocurrido y decidir cómo resolverlo: bien combinando los cambios o eligiendo uno de ellos.

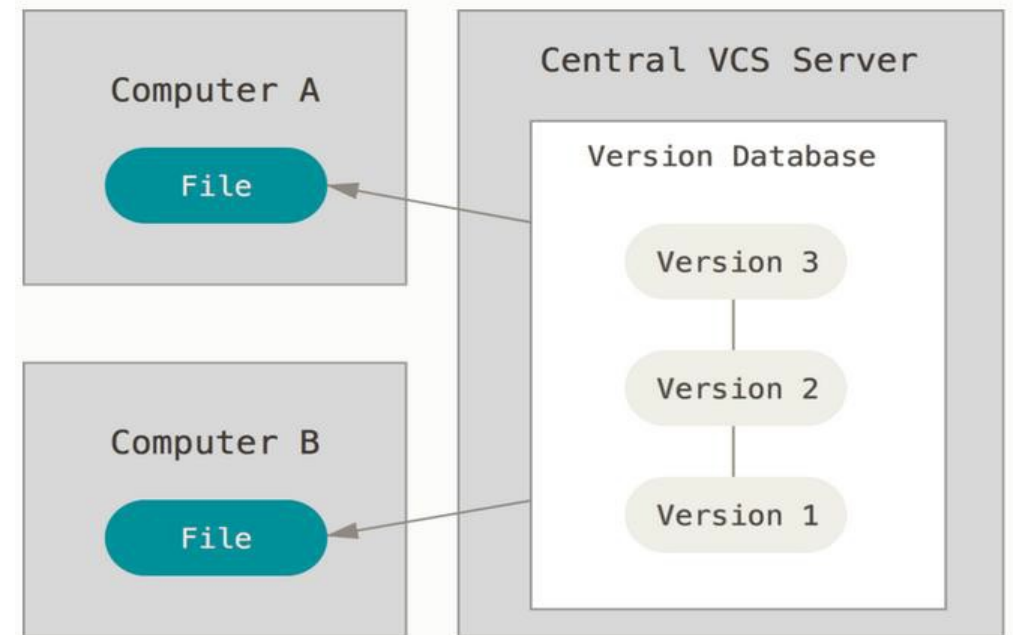
TIPOS DE REPOSITORIOS

- **Locales:** la información se guarda en diferentes directorios en función de sus versiones. Toda la gestión recae sobre el responsable del proyecto y no se dispone de herramientas que automaticen el proceso. Es viable para pequeños proyectos donde el trabajo es desarrollado por un único programador.



TIPOS DE REPOSITORIOS

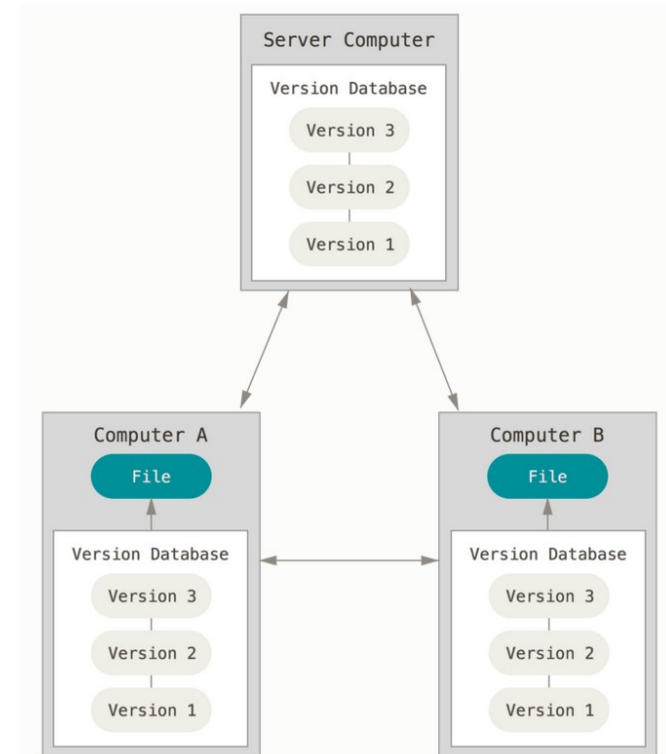
- **Centralizados:** existe un repositorio centralizado de todo el código en el servidor y el equipo de desarrollo dispone de herramientas para acceder desde su propio ordenador a este servidor, recuperar el proyecto, realizar cambios en su equipo local y enviarlos al servidor. Se facilitan las tareas administrativas a cambio de reducir flexibilidad, pues todas las decisiones fuertes (como crear una nueva rama) necesitan la aprobación del responsable. Además, si se produce un fallo en el servidor, se pierde toda la información del proyecto, teniendo que recurrir a las copias de seguridad. Algunos ejemplos de herramientas en sistemas centralizados son CVS y Subversion.



TIPOS DE REPOSITORIOS

- **Distribuidos:** aunque hay un repositorio central, cada usuario tiene su propia copia del repositorio completo, de modo que, ante un fallo en una copia, se podría recuperar la información de las otras copias. Antes de realizar cualquier cambio, actualiza las últimas versiones de todos los EC y trabaja en local. No es necesario tomar decisiones centralizadamente, puesto que hasta que no ha probado todo, no sube los cambios realizados al repositorio central. Los distintos repositorios pueden intercambiar y mezclar revisiones entre ellos. El rol de todos los equipos es de igual a igual y los cambios se pueden sincronizar entre cada par de copias disponibles. Aunque técnicamente todos los repositorios tienen la posibilidad de actuar como punto de referencia; habitualmente funcionan siendo uno el repositorio principal y el resto asumiendo un papel de clientes sincronizando sus cambios con éste.

Ejemplos: Git y Mercurial.



FUNCIONAMIENTO DE REPOSITORIOS

Todos los sistemas de control de versiones se basan en disponer de un repositorio. Este repositorio contiene el historial de versiones de todos los elementos gestionados.

Cada uno de los usuarios puede crearse una copia local duplicando el contenido del repositorio para permitir su uso. Es posible duplicar la última versión o cualquier versión almacenada en el historial.

Este proceso se suele conocido como check out se puede hacer de dos formas.

FUNCIONAMIENTO DE REPOSITORIOS

- **Exclusivos.** En este tipo de proceso, para poder realizar un cambio es necesario marcar en el repositorio el elemento que se desea modificar y el sistema se encargará de impedir que otro usuario pueda modificar dicho elemento.
- **Colaborativos.** En este tipo, cada usuario se descarga la copia, la modifica, y el sistema automáticamente combina las diversas modificaciones. El principal problema es la posible aparición de conflictos que deban ser solucionados manualmente o las posibles inconsistencias que surjan al modificar el mismo fichero por varias personas no coordinadas

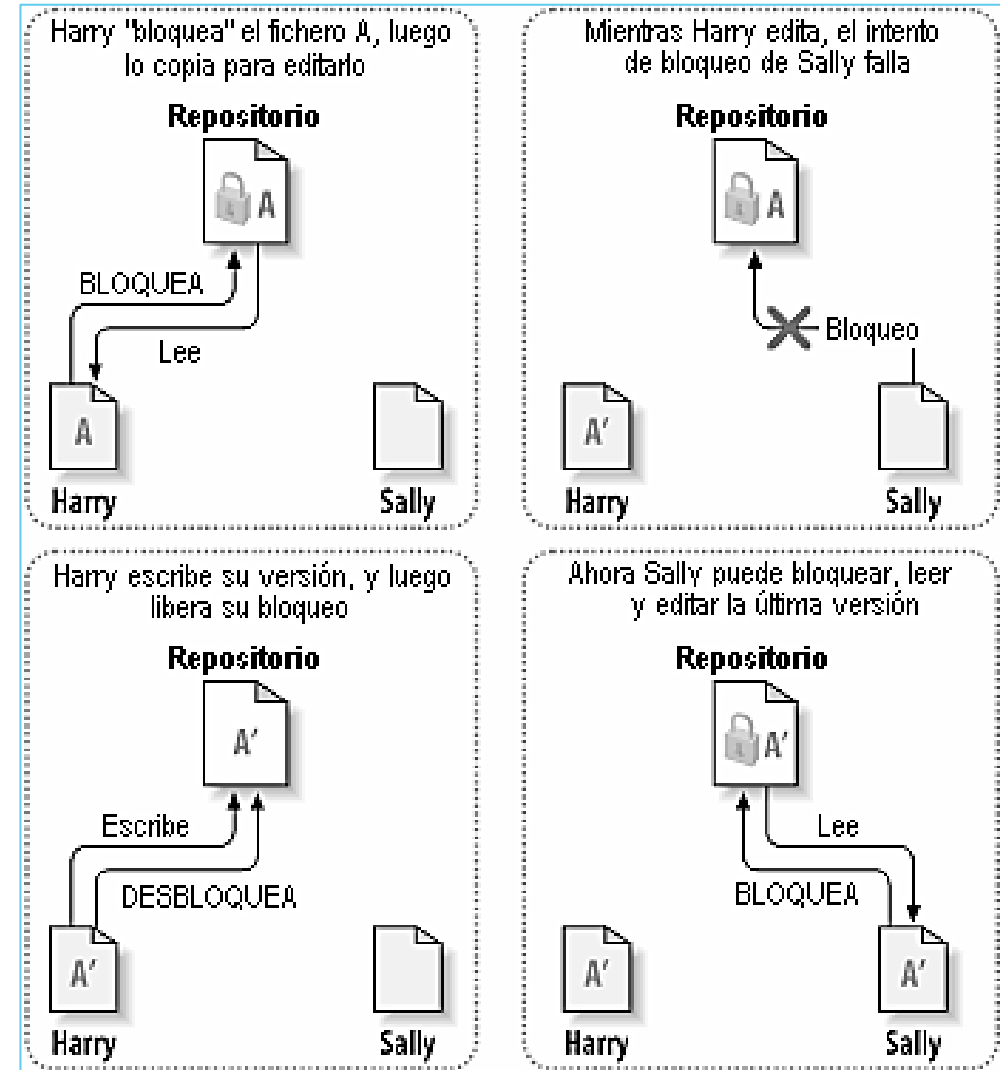
En cualquiera de los casos, tras realizar la modificación es necesario actualizar el repositorio con los cambios realizados (**commit**).

MODELOS DE VERSIONADO

Modelo bloquear-modificar-desbloquear

Sólo permite que una persona toque el código a la vez.

Hasta que esa persona no ha subido los cambios al repositorio, nadie más puede hacer checkout de ese mismo contenido y trabajar sobre él.



MODELOS DE VERSIONADO

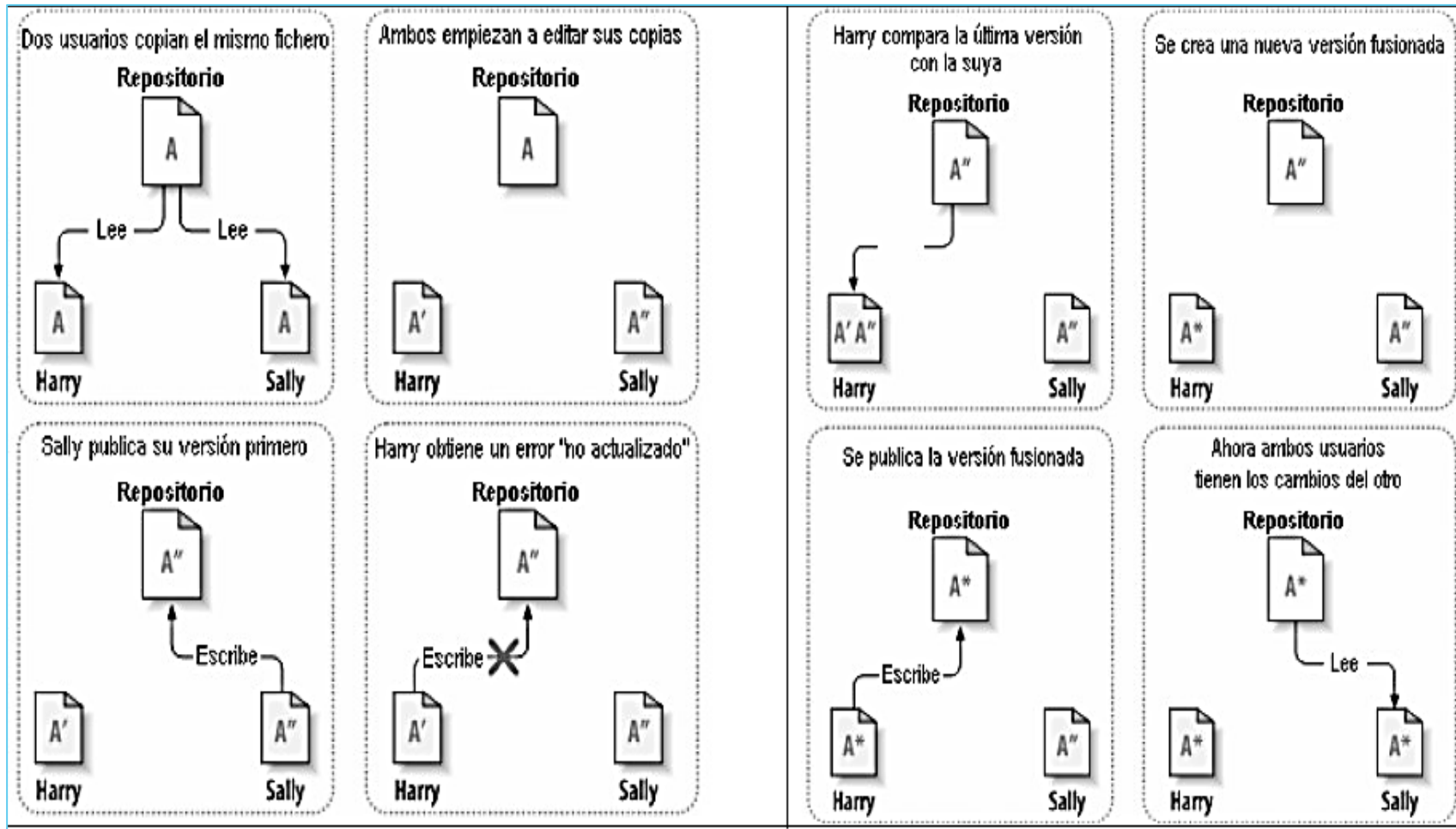
Modelo copiar- modificar -fusionar

Subversion, CVS y otros sistemas de control de versiones utilizan un modelo copiar-modificar-fusionar como alternativa a bloquear. En este modelo, el cliente de cada usuario lee el repositorio y crea una copia de trabajo personal del archivo o del proyecto.

Luego, los usuarios trabajan en paralelo, modificando sus copias privadas. Finalmente, las copias privadas se fusionan juntas en una nueva versión final. El sistema de control de versiones a menudo ofrece ayuda en la fusión, pero al final la persona es la responsable de hacer que ocurra correctamente.

MODELOS DE VERSIONADO

Modelo copiar- modificar -fusionar



CONFLICTO

¿Pero qué ocurre en este segundo modelo si los cambios de Sally **sí** se superponen a los cambios de Harry? ¿Qué hacemos entonces?

Esta situación se denomina un **conflicto**, y normalmente no supone mucho problema. Cuando Harry le pide a su cliente que fusione los últimos cambios del repositorio en su copia de trabajo, su copia del archivo A se marca de alguna forma como que está en un estado de conflicto: él será capaz de ver ambos conjuntos de cambios conflictivos, y manualmente podrá elegir entre ellos. Hay que tener en cuenta que el software no puede resolver conflictos automáticamente; sólo los humanos son capaces de entender y hacer las elecciones necesarias de forma inteligente. Una vez que Harry haya resuelto manualmente los cambios que se superponían (probablemente hablando con Sally para poder tomar una decisión), puede guardar de forma segura el archivo fusionado al repositorio.

SUBVERSION

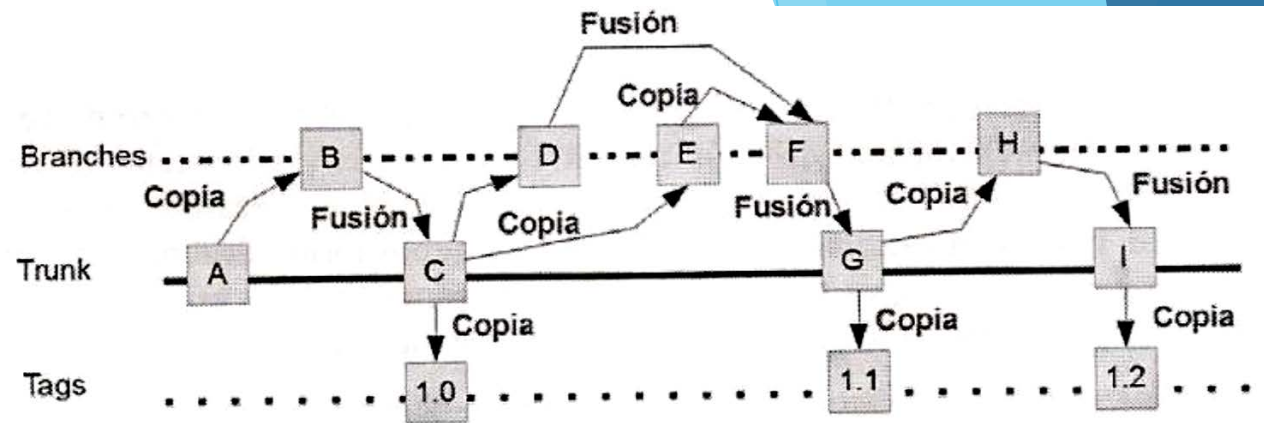
Es una herramienta multiplataforma de código abierto para el control de versiones.

Usa una base de datos de central como repositorio, que contiene los archivos cuyas versiones y respectivas historias se controlan. El repositorio actúa como un servidor de ficheros, con la capacidad de recordar todos los cambios que se hacen tanto en sus directorios como en sus ficheros.

El proyecto debe tener estructura de árbol: el tronco (trunk) donde está la línea principal de su desarrollo, las ramas (branches) en la que se añadirán nuevas funciones o se corregirán errores y sus etiquetas (tags) para marcar situaciones importantes, o versiones finalizadas.

El ciclo de vida de un proyecto software en Subversion podría un aspecto similar al que se muestra en la siguiente diapositiva.

SUBVERSION



A. Se parte del desarrollo inicial, en el trunk.

B. Se crea una rama porque hay que añadir una nueva funcionalidad, por ejemplo.

C. Mientras tanto se ha corregido un error en la línea de desarrollo principal y hemos llegado a C. Una vez que está lista la rama B se fusiona en el trunk. Cuando está lista esta versión libre de errores y con la nueva funcionalidad terminada se crea la primera versión disponible para el público (etiqueta - 1.0).

D. Después de salir la primera versión, se han detectado nuevos errores, y se necesita añadir nuevas funcionalidades. Entonces se crea una rama para desarrollar una nueva funcionalidad.

E. Se ha creado otra rama porque se necesitan otras funcionalidades de las que se encargarán otros miembros del equipo.

F. Se realiza la fusión de las dos ramas, primero se realiza una copia de una de ellas (E) y luego se fusiona con la otra (D).

G. Mientras tanto se han ido solucionando errores y desarrollando más código en la línea principal, de modo que las nuevas funcionalidades de F se incorporan al tronco principal en G. Una vez que está lista esta revisión se crea una nueva versión para el público (etiqueta 1.1).

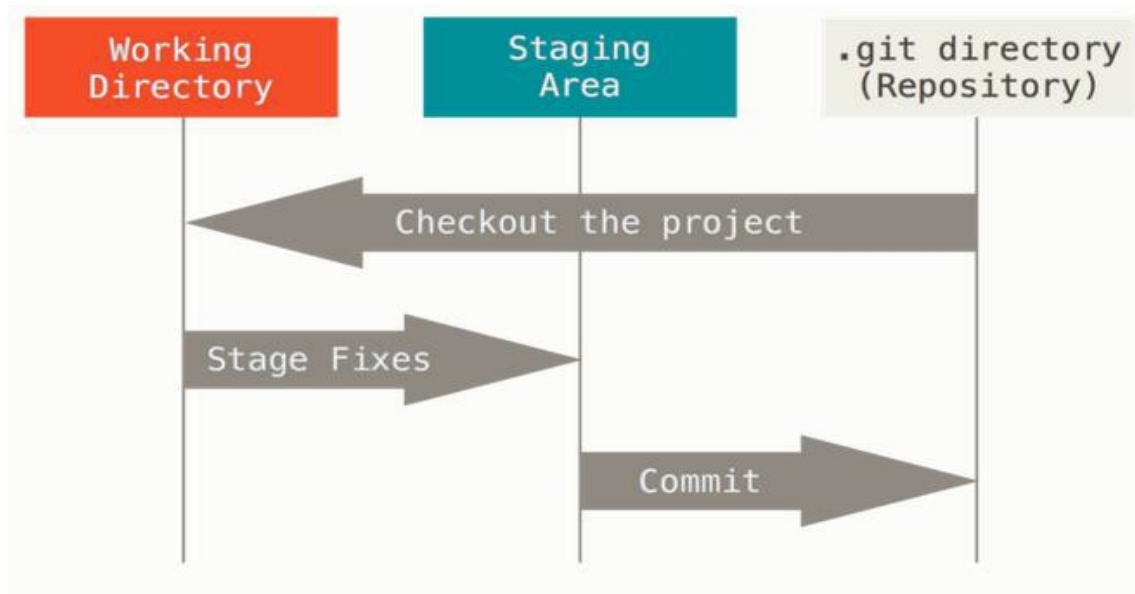
H. Se han detectado nuevos errores y además se necesitan añadir nuevas funcionalidades. Se crea una nueva rama para desarrollar una nueva funcionalidad.

I. Se incorporan las nuevas funcionalidades de H al tronco principal donde ya se han corregido los errores y se ha llegado a I. Se crea una nueva versión para el público (etiqueta 1.2).

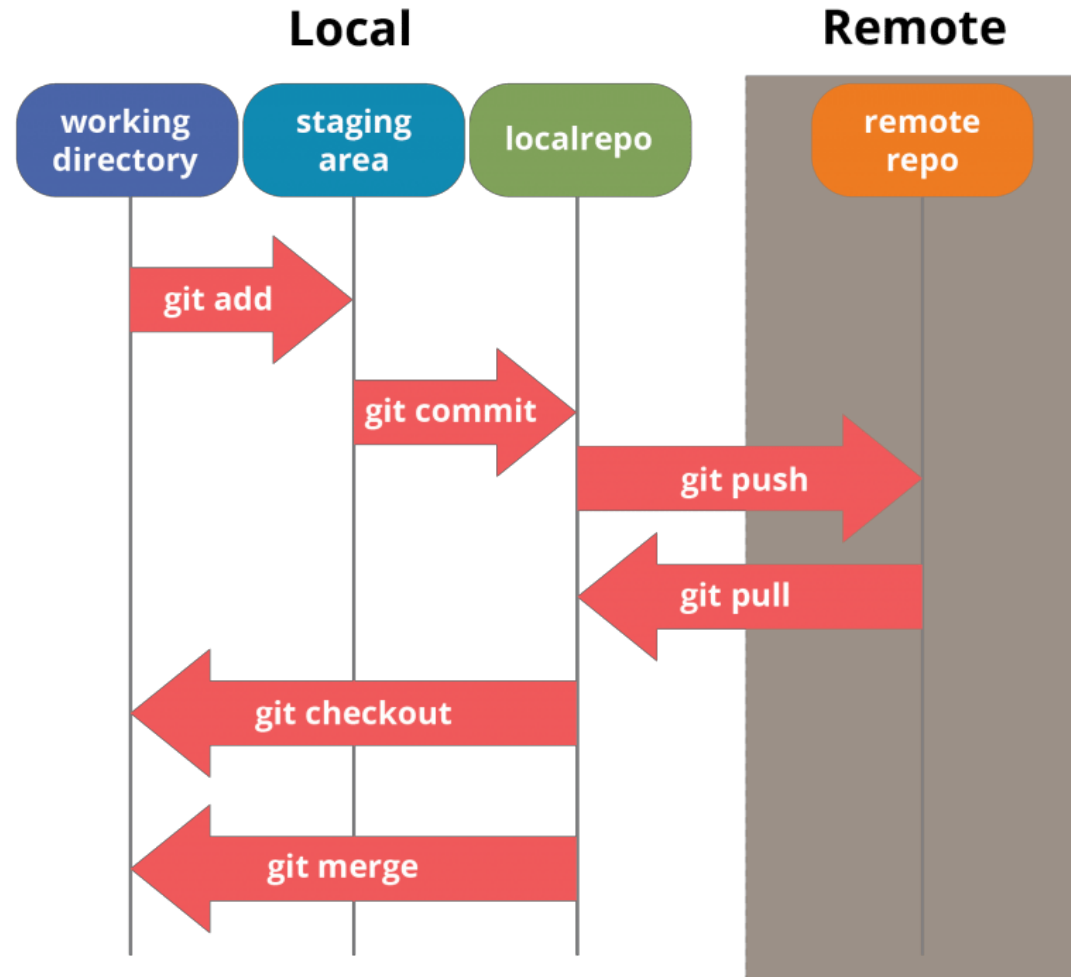
GIT

Git tiene tres estados principales en los que pueden residir sus archivos: confirmado (committed), modificado (modified), y preparado (staged):

- ▶ Confirmado: significa que los datos están almacenados de manera segura en tu base de datos local.
- ▶ Modificado: significa que has modificado el archivo pero todavía no lo has confirmado a tu base de datos.
- ▶ Preparado: significa que has marcado un archivo modificado en su versión actual para que vaya en tu próxima confirmación.



GIT - Estructura



GIT – Ramas (Branch)

La rama por defecto de Git es la rama main (master). Con el primer commit de cambios que realicemos, se creará esta rama principal master apuntando a dicho commit.

Las ramas en Git son líneas independientes de desarrollo. Se pueden entender como una forma de solicitar un nuevo directorio de trabajo, un nuevo ambiente de trabajo o un nuevo historial en el proyecto.

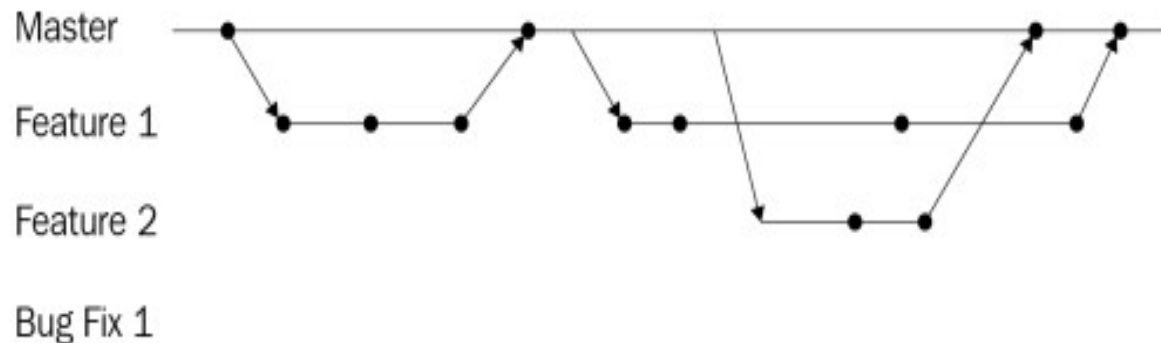
Todos los commits son registrados en el historial de la rama principal, creando una bifurcación en el historial del proyecto.

La fusión es la forma que tiene Git de volver a unir un historial bifurcado. El comando `git merge` permite tomar las líneas independientes de desarrollo creadas por `git branch` e integrarlas en una sola rama.

GIT – Ramas (Flujo GitHub)

Es una estrategia de ramificación simple, hay una rama main (master) que siempre debe estar en un estado funcional. No se permiten incorporar funcionalidad sin terminar en la rama maestra.

Si se desea comenzar a trabajar en una nueva característica o corregir errores, se debe crear una nueva rama desde la rama main (master). Solo cuando se haya terminado por completo con ese trabajo se debe fusionar esta rama nuevamente con la rama master.

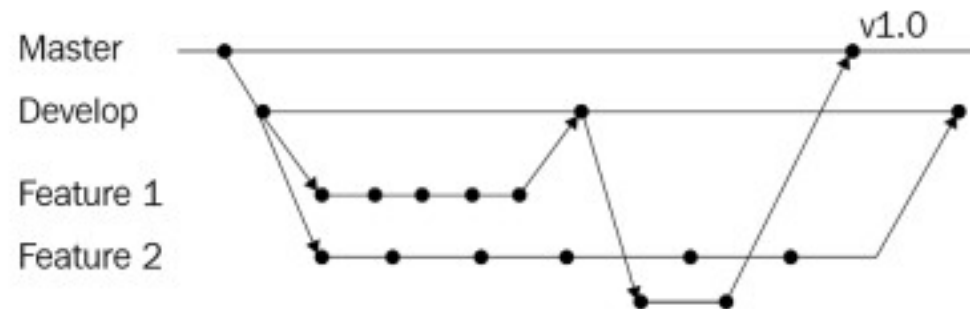


GIT – Ramas (Flujo GitFlow)

GitFlow tiene dos ramas principales que son la rama de desarrollo y la rama main (master). En la rama de desarrollo se integran todos los progresos, se combinan las nuevas funciones y se realizan pruebas de integración. Sólo debe contener trabajos que creamos que están listo para ser liberados.

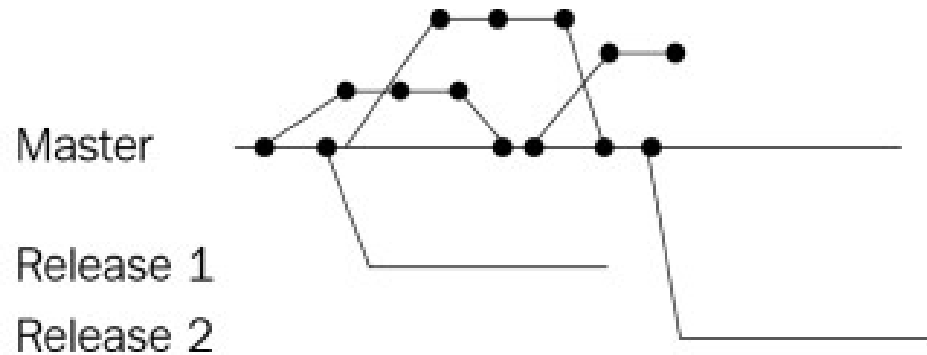
Desde la rama de desarrollo, se crean las ramas de características-funcionalidad donde se comienza a trabajar en las nuevas funcionalidades. Solo cuando una característica esté lista se debe fusionar con la rama de desarrollo.

Cuando se desea lanzar una nueva versión de la aplicación, se crea una rama de lanzamiento desde la rama de desarrollo. Se pueden realizar pruebas finales del código de esta rama y realizar una o más correcciones de errores si es necesario. Cuando esta todo correcto, se fusiona esta rama con main (master) y se etiqueta la versión



GIT – Ramas (Flujo Release)

Se basa en trabajar con ramas temáticas de corta duración que se crean y fusionan en la rama main (master). La diferencia es que el código que está en la rama main (master) no es el que se implementa en producción. En cambio, cada vez que es necesario lanzar una nueva versión del producto, se crea una nueva rama de master con el nombre lanzamiento-{versión}. El código de esta rama luego se implementa en producción. Una vez que se implementa una nueva rama de versión, se puede ignorar la anterior. Este da como resultado el siguiente flujo:



GITHUB

GitHub es un servicio basado en la nube que aloja un sistema de control de versiones (VCS) llamado Git. Éste permite a los desarrolladores colaborar y realizar cambios en proyectos compartidos, a la vez que mantienen un seguimiento detallado de su progreso.

Aloja más de 100 millones de repositorios, la mayoría de los cuales son proyectos de código abierto. Esta estadística revela que GitHub se encuentra entre los clientes Git GUI más populares y es utilizado por varios profesionales y grandes empresas.

Es una plataforma de gestión y organización de proyectos basada en la nube que incorpora las funciones de control de versiones de Git.

Los usuarios de GitHub pueden rastrear y gestionar los cambios que se realizan en el código fuente en tiempo real, a la vez que tienen acceso a todas las demás funciones de Git disponibles en el mismo lugar.

Para poder trabajar en GitHub es necesario registrar un usuario en la nube. (<https://github.com>)

TRABAJAR CON GITHUB EN ECLIPSE - PAT

Dado que GitHub ya no permite el uso de contraseñas tradicionales para autenticar aplicaciones de terceros como Eclipse, es necesario generar un Token de Acceso Personal (PAT). Este token funcionará como nuestra nueva contraseña segura.

Para crear un PAT haremos lo siguiente en la web de GitHub:

1. Iniciaremos sesión en nuestra cuenta de GitHub.
2. Haremos clic en nuestra foto de perfil (arriba a la derecha) y seleccionaremos **Settings** (Configuración).
3. Iremos al final del menú lateral izquierdo y haremos clic en **Developer settings**.
4. En el menú izquierdo, desplegaremos **Personal access tokens** y hacemos clic en **Tokens** (classic).
5. Hacemos clic en el botón Generate new token (y seleccionamos classic si el sistema nos da a elegir).
6. Configuraremos los permisos:
 1. Note: Escribiremos un nombre descriptivo para saber para qué es (por ejemplo, "Eclipse EGit").
 2. Expiration: Elegiremos cuándo caducará (por seguridad, se recomienda 30, 60 o 90 días).
 3. Select scopes (Permisos): Marcaremos la casilla principal llamada repo. Esto da a Eclipse el control necesario sobre nuestros repositorios para poder subir (push) y descargar (pull) código.
 4. Vamos al final y pulsamos Generate token.

¡MUY IMPORTANTE! Copia la contraseña que aparece en pantalla y guárdalo bien. GitHub no nos lo volverá a mostrar. Este es el código que debemos pegar en el campo "Password" de Eclipse al conectar nuestro proyecto remoto.

CREAR UN REPOSITORIO en GITHUB

Dado que GitHub es un servicio basado en la nube que aloja repositorios Git, el proceso consta de dos fases: primero preparar el espacio en la nube y luego decirle a nuestro entorno local (Eclipse) dónde está ese espacio.

Antes de enviar el código desde Eclipse, necesitamos un contenedor vacío en internet que lo reciba. Para crear este repositorio haremos los siguientes pasos.

1. Entramos en nuestra cuenta de GitHub en el navegador.
2. Nuevo repositorio: Haz clic en el botón verde "New" (o en el icono "+" en la esquina superior derecha y selecciona "New repository").
3. Configuración:
 1. Asigna un nombre al repositorio (suele coincidir con el nombre de tu proyecto en Eclipse).
 2. Elige si será Público (visible para todos) o Privado (solo para ti y tus colaboradores).

Importante: NO marques la casilla "Add a README file" ni añadas un archivo .gitignore si ya tienes un proyecto creado en Eclipse. Queremos que el repositorio remoto nazca completamente vacío para evitar conflictos al subir nuestro código local.
4. Crear: Haz clic en "Create repository". GitHub nos mostrará una pantalla con instrucciones. Copia la URL web del nuevo repositorio (suele tener el formato <https://github.com/tu-usuario/tu-proyecto.git>).

TRABAJAR CON GITHUB EN ECLIPSE

EGit es un proveedor del equipo Eclipse para el sistema de control de versiones Git. Git es un SCM distribuido, lo que significa que cada desarrollador tiene una copia completa de todo el historial de cada revisión del código, lo que hace que las consultas al historial sean muy rápidas y versátiles.

Eclipse trae de serie una componente denominada EGit que proporciona una interfaz bastante completa de las operaciones con Git.

Si no estuviese disponible, para instalar el plugin iremos a Help>Eclipse Marketplace Buscar eGit y lo instalamos.

Para acceder a la vista de Egit en Eclipse iremos al menú Window > Show View> Git Repositories.

Para trabajar con Git, buscamos la perspectiva de Git y la abrimos (Open perspective).

ASOCIAR UN REP GITHUB A ECLIPSE

Ya tenemos nuestro repositorio en Github y nuestro proyecto en Eclipse. También hemos generado nuestro PAT que nos permitirá conectar ambos de forma segura. Para lograr esto seguiremos los siguientes pasos:

FASE 1: Convertir el proyecto local en un Repositorio Git

1. En nuestro proyecto en Eclipse, haremos clic derecho. Vamos a **Team > Share Project**.
2. Pulsamos en **Git** y en **Next**.
3. Seleccionamos nuestro proyecto de la lista. Después pulsamos en **Create Repository** y luego en **Finish**.

ASOCIAR UN REP GITHUB A ECLIPSE

FASE 2: Hacer el primer "Commit" (Guardado local)

Antes de subir nada a nuestro repositorio, tenemos que empaquetar los archivos que queremos subir.

1. Hacemos clic derecho sobre el proyecto > **Team > Commit...** Se abrirá una vista nueva llamada **Git Staging** (normalmente en la parte inferior de la pantalla).
2. En la caja superior izquierda llamada **Unstaged Changes**, veremos nuestros archivos (como Calculadora.java, .classpath, .project, etc.). Seleccionamos los archivos que queremos subir y los arrastramos a la caja inferior llamada **Staged Changes** (o usamos el botón con el icono de un doble "más" ++ para añadirlos todos).
3. En la caja de la derecha (**Commit Message**), escribimos un mensaje descriptivo. Por ejemplo: "Commit inicial del proyecto".
4. Hacemos clic en el botón **Commit** (Ojo: en "Commit", no en "Commit and Push" por ahora, para ir paso a paso).

ASOCIAR UN REP GITHUB A ECLIPSE

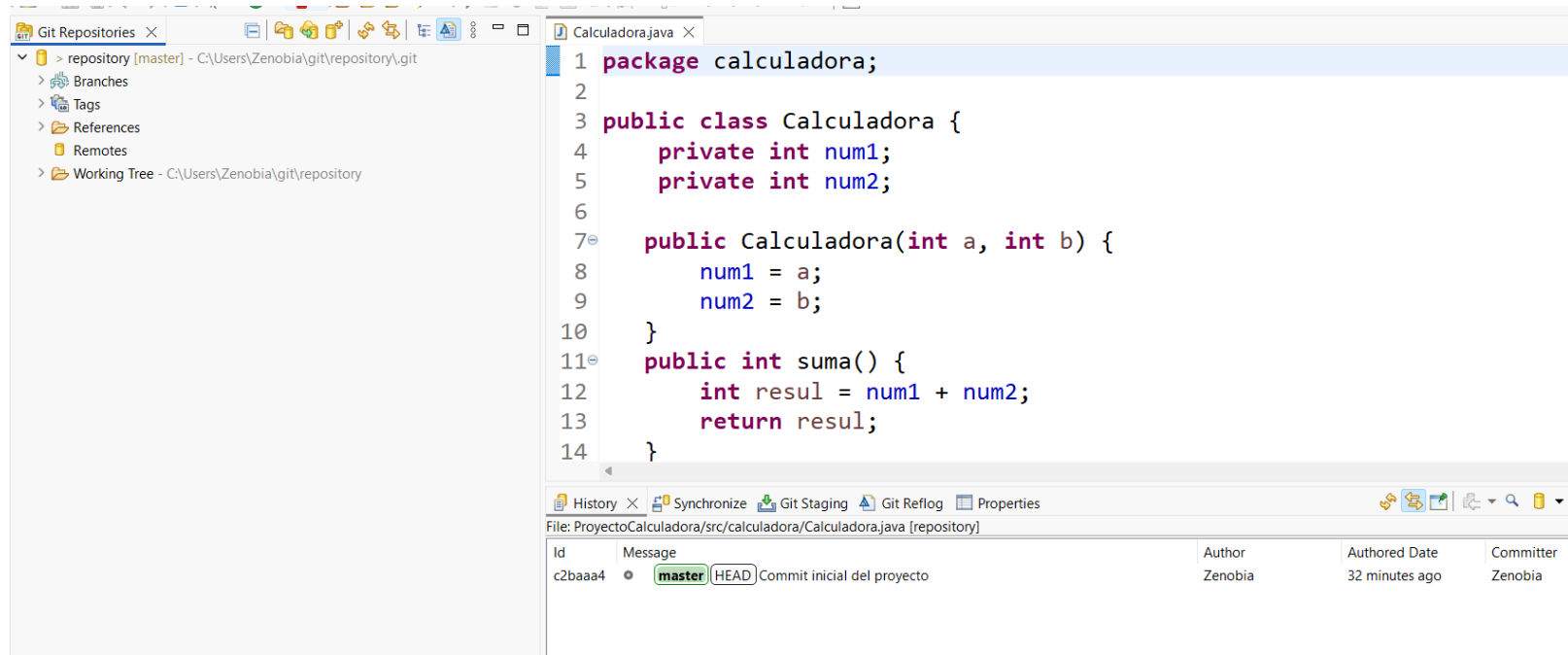
FASE 3: Vincular con GitHub y Subir (Push)

Ahora vamos a decirle a Eclipse dónde está ese repositorio vacío en internet y enviarle el paquete que acabamos de crear.

1. Copiamos la URL del repositorio vacío en GitHub.com (suele tener el formato <https://github.com/tu-usuario/tu-proyecto.git>).
2. En Eclipse, hacemos clic derecho en el proyecto > **Team > Remote > Push...** Se abrirá una ventana de configuración:
 1. URI: Pegamos la URL que acabamos de copiar de GitHub. (Eclipse rellenará automáticamente los campos de Host y Repository path).
 2. Authentication:
 1. User: nuestro nombre de usuario de GitHub.
 2. Password: Aquí NO va nuestra contraseña, sino el Personal Access Token (PAT) que generamos previamente en GitHub.
 3. Marcamos la casilla **Store in Secure Store** para que Eclipse recuerde el token y no lo pida cada vez y pulsamos **Next**.
3. En la siguiente pantalla, le diremos a Eclipse la rama que queremos subir:
 1. En **Source ref** seleccionamos nuestra rama actual (suele llamarse refs/heads/master o refs/heads/main).
 2. Pulsamos **Add Spec** y aparecerá una línea nueva en la caja inferior de **Specifications for push**. Pulsamos **Finish**.

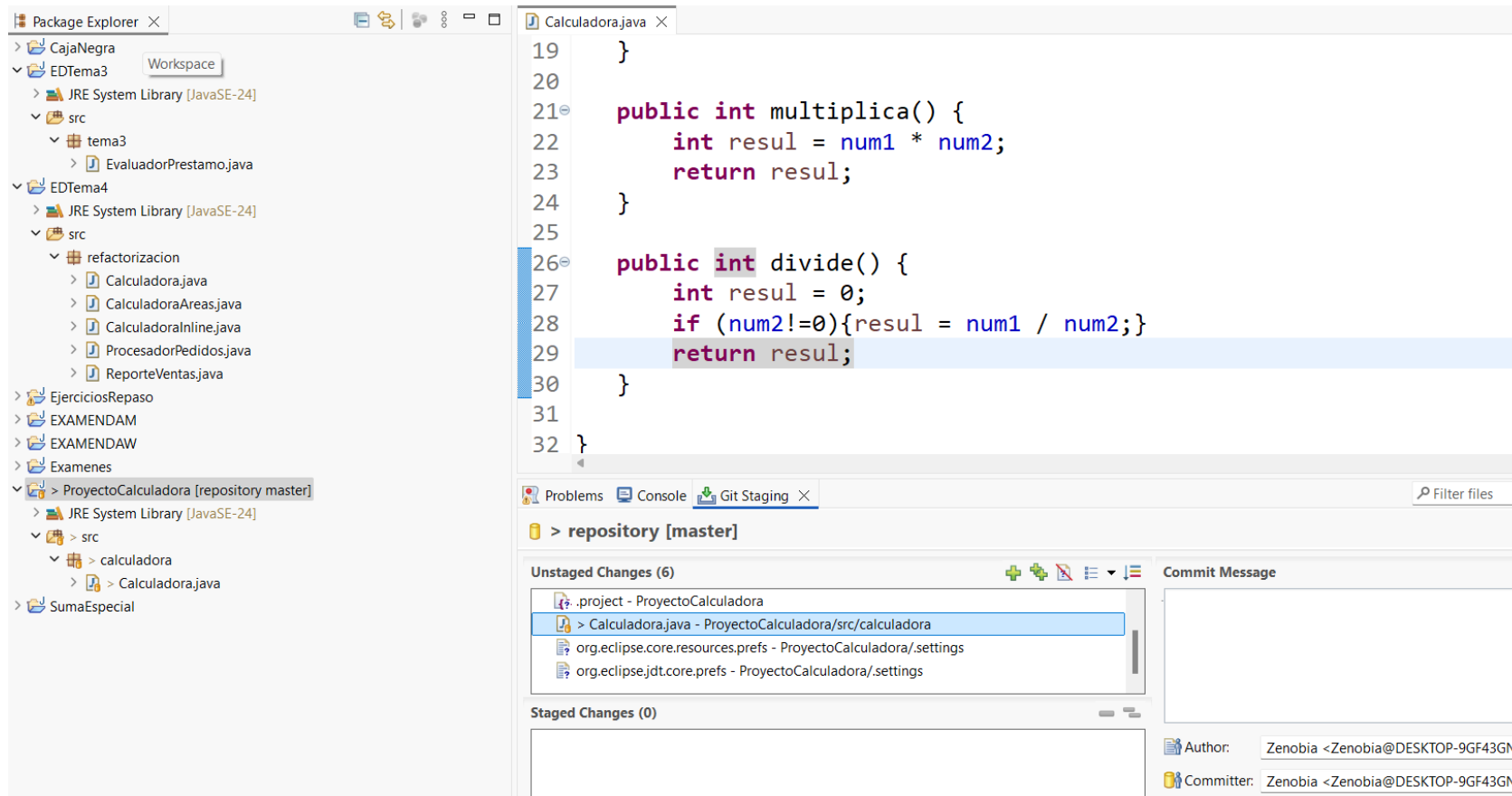
USO INTEGRADO DE GIT y ECLIPSE

- ▶ Accedemos a la perspectiva de GIT: (menú Window > Open Perspective > Other > "Git").



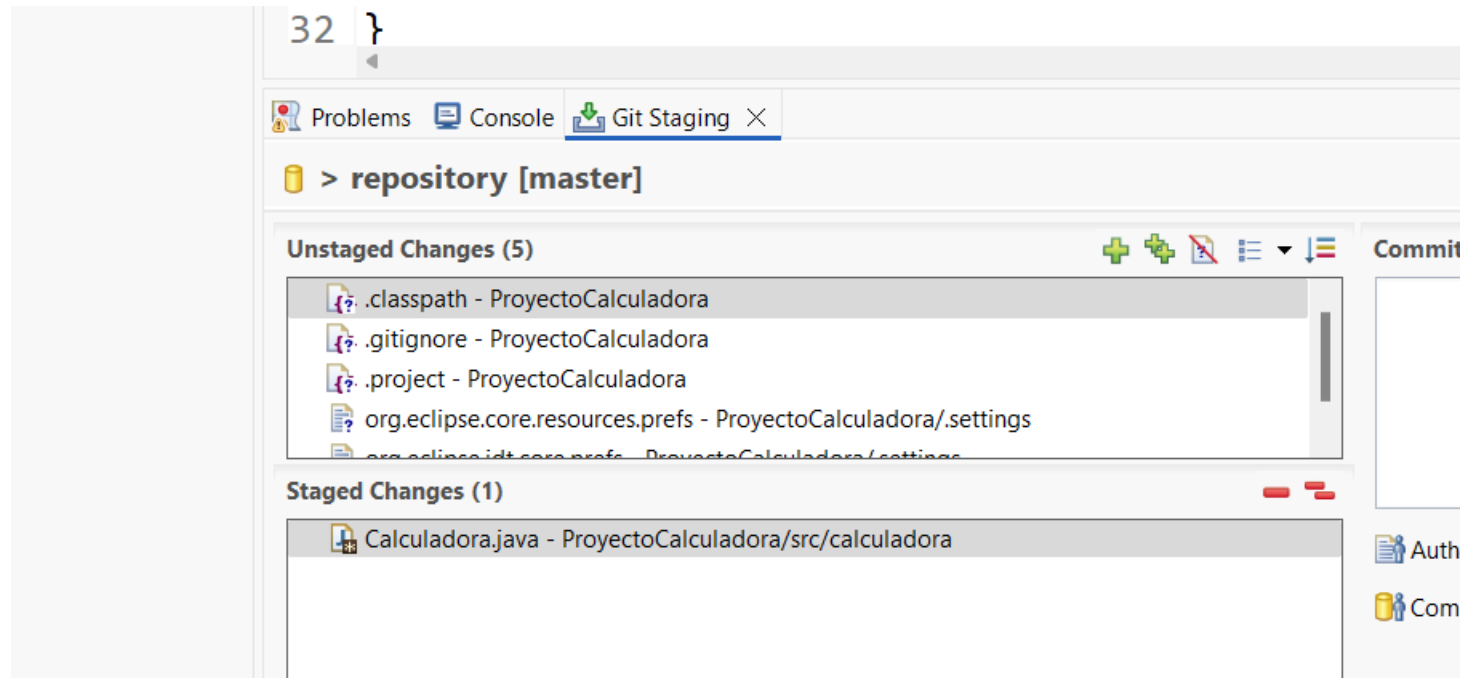
USO INTEGRADO DE GIT y ECLIPSE

- ▶ Al realizar cambios en el proyecto, y guardar, en la vista Git Staging se pueden ver todos los cambios que se han ido produciendo.



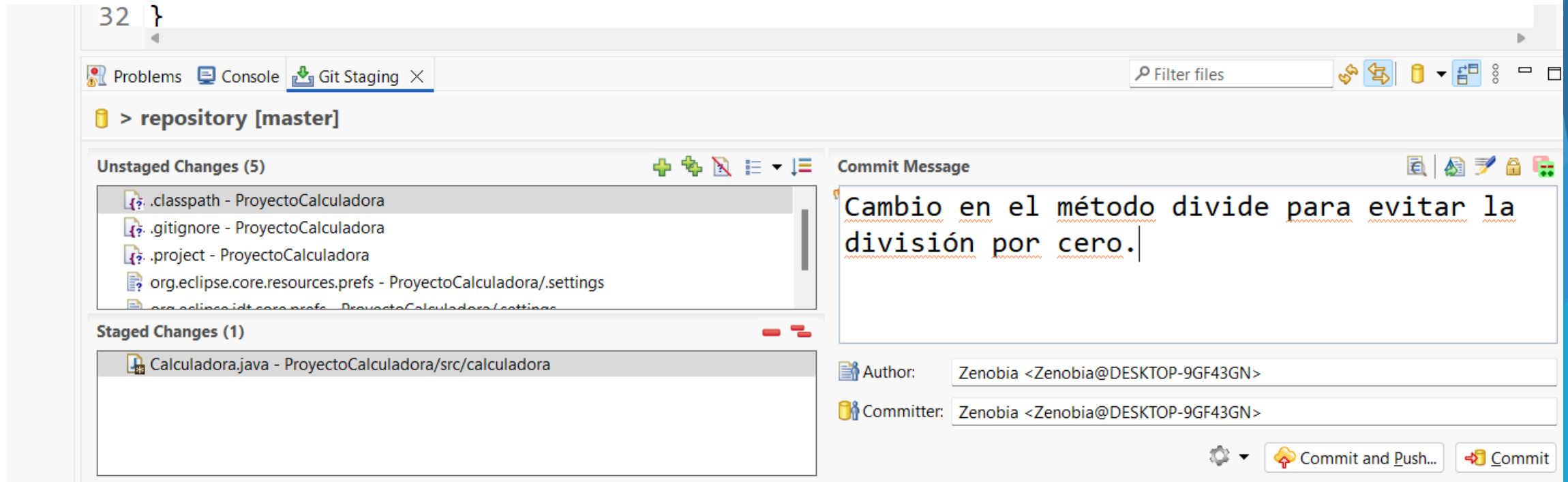
USO INTEGRADO DE GIT y ECLIPSE

- Como ya hemos visto, para guardar los distintos cambios en el repositorio local a través de un commit, se pueden arrastrar los cambios de Unstaged Changes a Staged Changes o bien añadirlos a través de la interfaz con los dos botones.



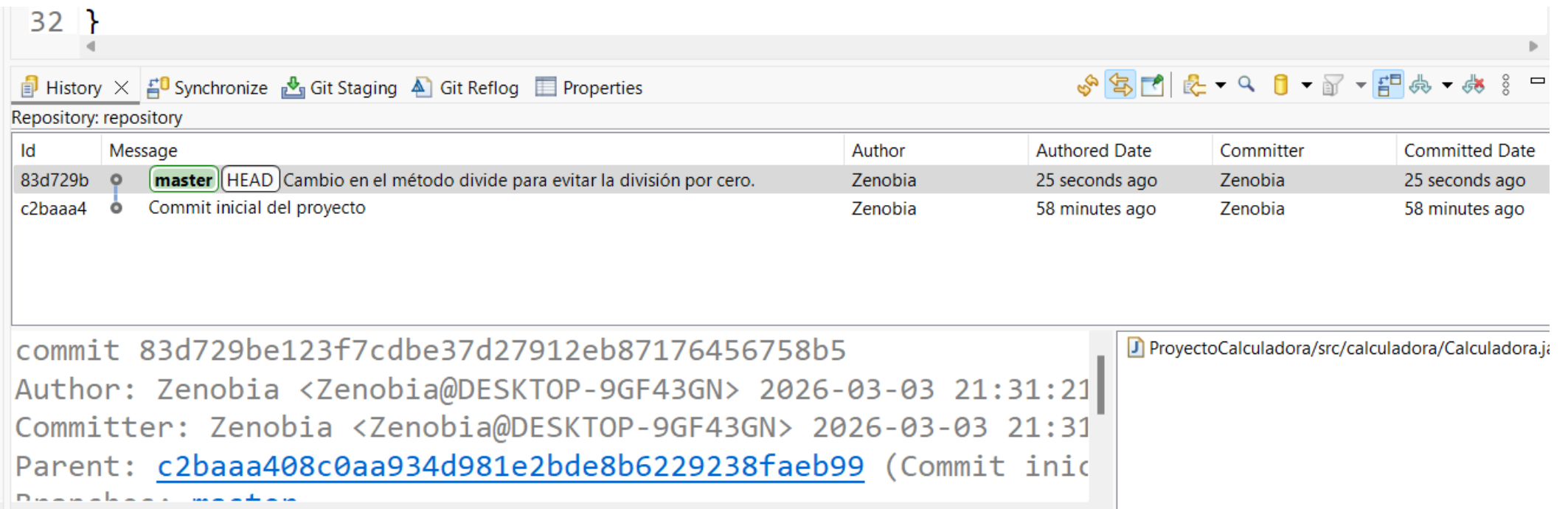
USO INTEGRADO DE GIT y ECLIPSE

- ▶ Cuando se han seleccionado los cambios que se quieren guardar y se ha establecido un mensaje descriptivo de todos los cambios a incluir, se habilitan los botones de Commit y Push y de Commit.



USO INTEGRADO DE GIT y ECLIPSE

- ▶ Al hacer el commit podemos comprobar que se hizo correctamente en la pestaña History.



32 }

History X Synchronize Git Staging Git Reflog Properties

Repository: repository

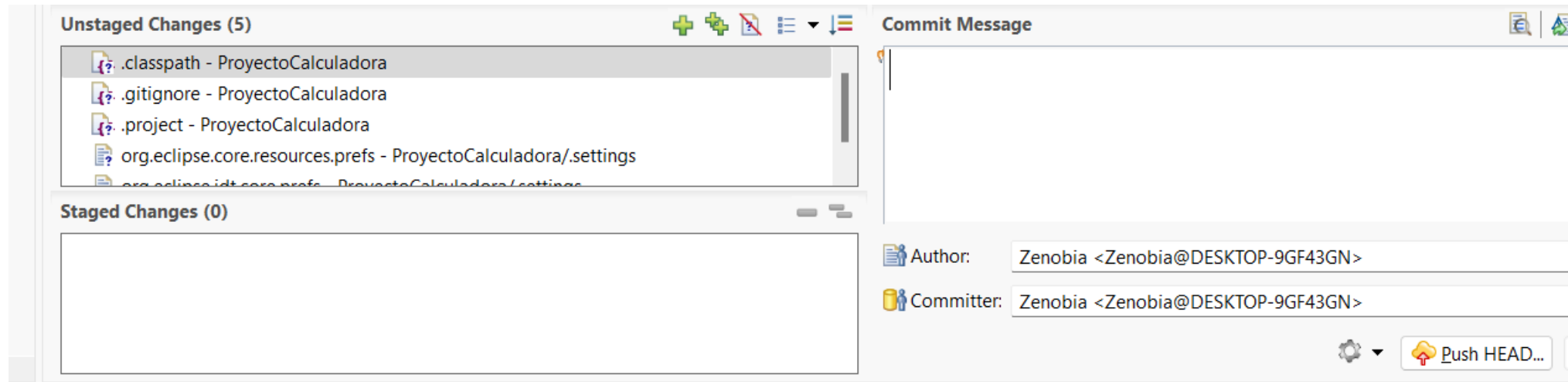
Id	Message	Author	Authored Date	Committer	Committed Date
83d729b	master HEAD Cambio en el método divide para evitar la división por cero.	Zenobia	25 seconds ago	Zenobia	25 seconds ago
c2baaa4	Commit inicial del proyecto	Zenobia	58 minutes ago	Zenobia	58 minutes ago

commit 83d729be123f7cdbe37d27912eb87176456758b5
Author: Zenobia <Zenobia@DESKTOP-9GF43GN> 2026-03-03 21:31:21
Committer: Zenobia <Zenobia@DESKTOP-9GF43GN> 2026-03-03 21:31:21
Parent: [c2baaa408c0aa934d981e2bde8b6229238faeb99](#) (Commit inicial del proyecto)

ProyectoCalculadora/src/calculadora/Calculadora.java

USO INTEGRADO DE GIT y ECLIPSE

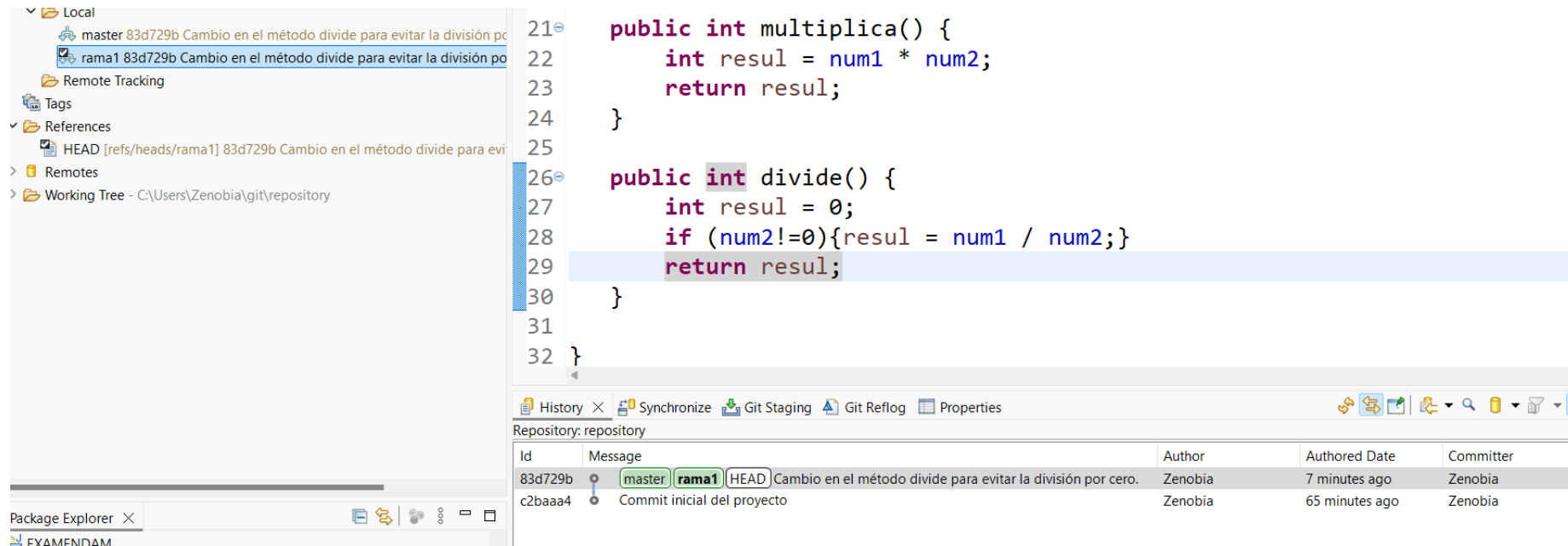
- Y se habilita la opción de Push HEAD



- Relleno la información y hago el push, y puedo comprobar los cambios en GitHub consultando las ramas.

USO INTEGRADO DE GIT y ECLIPSE

► Para crear una nueva rama en el repositorio: Botón derecho en el proyecto y vamos a Team > Switch to > New Branch

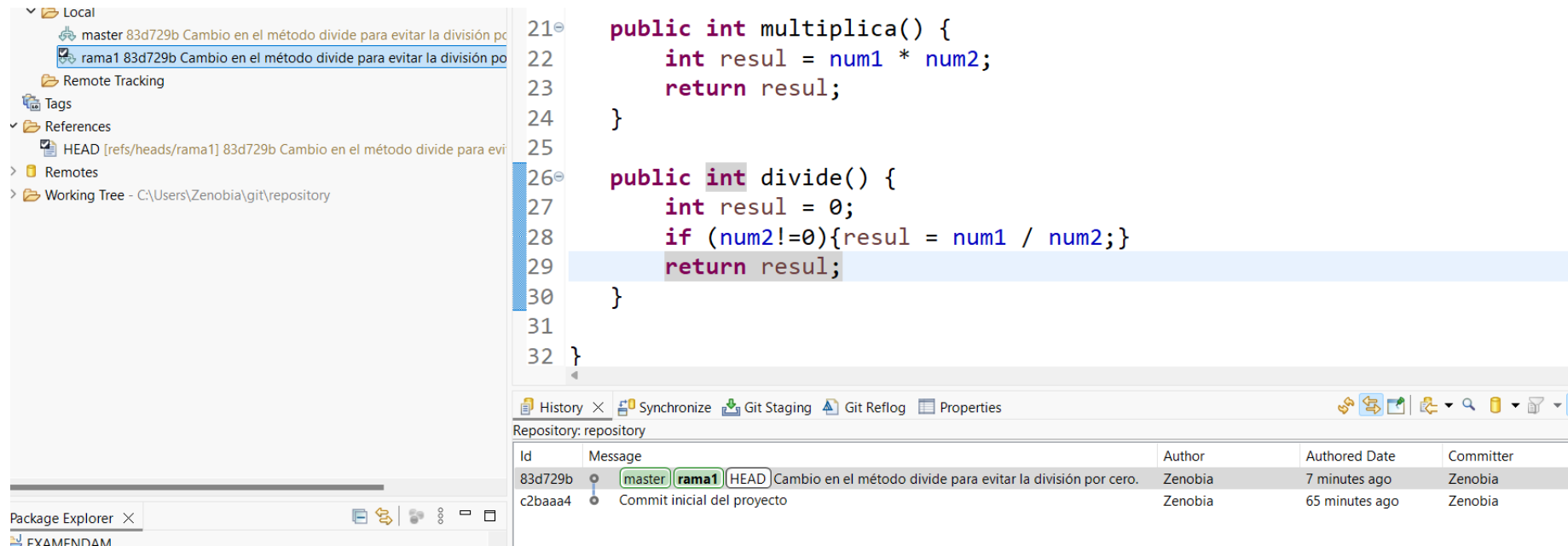


```
21 public int multiplica() {
22     int resul = num1 * num2;
23     return resul;
24 }
25
26 public int divide() {
27     int resul = 0;
28     if (num2!=0){resul = num1 / num2;}
29     return resul;
30 }
31
32 }
```

Id	Message	Author	Authored Date	Committer
83d729b	Cambio en el método divide para evitar la división por cero.	Zenobia	7 minutes ago	Zenobia
c2baaa4	Commit inicial del proyecto	Zenobia	65 minutes ago	Zenobia

USO INTEGRADO DE GIT y ECLIPSE

- Para crear una nueva rama en el repositorio: Botón derecho en el proyecto y vamos a Team > Switch to > New Branch



USO INTEGRADO DE GIT y ECLIPSE

Si hago un cambio en la rama 1 y hago commit, cambio a la rama master ese cambio no es visible en ella. Si quiero aplicar ese cambio a la rama principal, haré un merge.

Voy a la rama que recibirá el cambio (master) y hago Team > Merge y le digo que lo haga desde la rama 1, y se aplicará al pulsar Merge.

